

The midterm project

1. *Data Preprocessing:*

(1) We know that we do not need the cause of the machine failure and the UID, Product ID, so we delete these lines.

```
# 删除无关的列
data = data.drop(['UDI', 'Product ID', 'TWF', 'HDF', 'PWF', 'OSF', 'RNF'], axis=1)
```

(2) Observing that there are characters in the line of Type, then transform them to numerical values, I use '0' means low, '1' means middle and '2' means high.

```
# 将 'Type' 列映射为数值
data['Type'] = data['Type'].map({'M': 1, 'L': 0, 'H': 2})
```

(3) We find that there are too many negative labels in the data-set and too less positive labels, if we use all of the data, then the F2 scores of the prediction of all the models will be too low because there are nearly no predictions which is positive. To deal with it, a simple idea is to randomly select some of the negative data.

```
print("原始正类样本数量:", np.sum(y == 1))
print("原始负类样本数量:", np.sum(y == 0))
```

原始正类样本数量: 339
原始负类样本数量: 9661

from the operation, we can choose a parameter(4%) which makes the number of selected negative data is close to the number of positive data.

```
# 分离正类和负类
X_negative = X[y == 0].reset_index(drop=True)
y_negative = y[y == 0].reset_index(drop=True)
X_positive = X[y == 1].reset_index(drop=True)
y_positive = y[y == 1].reset_index(drop=True)

# 随机选取 4% 的负类
negative_sample_size = max(1, int(0.04 * len(X_negative))) # 至少采样 1 个负类
if negative_sample_size > len(X_negative):
    raise ValueError("负类数据不足, 无法采样 4%")
negative_indices = np.random.choice(len(X_negative), size=negative_sample_size, replace=False)
X_negative_sampled = X_negative.iloc[negative_indices].to_numpy()
y_negative_sampled = y_negative.iloc[negative_indices].values # 使用 .values 确保索引有效

# 合并正类和抽样后的负类
X_sampled = np.vstack((X_positive.to_numpy(), X_negative_sampled))
y_sampled = np.hstack((y_positive.to_numpy(), y_negative_sampled))
```

To make the result more convincing, we make the data more random.

```
# 打乱数据顺序
indices = np.arange(len(y_sampled))
np.random.shuffle(indices)
X = X_sampled[indices]
y = y_sampled[indices]
```

Showing the consequence of selecting, now the numbers are near and we can do the next step.

```
print("采样后正类样本数量:", np.sum(y == 1))
print("采样后负类样本数量:", np.sum(y == 0))
```

采样后正类样本数量：339
采样后负类样本数量：386

(4) Normalization of the features and return X,y.

```
# 特征归一化 (最小-最大缩放)
X = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0)) # 归一化处理

return X, y # 返回 numpy 数组
```

2. **Feature Engineering:**

(1) Set X and y.

```
# 特征选择: 除了 'Machine failure' 列之外的所有特征
X = data.drop(columns=['Machine failure'])
y = data['Machine failure']
```

(2) Split train and test set.

```
# 划分数据集为训练集和测试集
def train_test_split(X, y, test_size=0.3, random_seed=42):
    np.random.seed(random_seed) # 设置随机种子, 保证结果可复现
    indices = np.arange(X.shape[0]) # 获取数据的索引
    np.random.shuffle(indices) # 随机打乱索引
    split_index = int((1 - test_size) * len(indices)) # 计算训练集大小
    train_idx, test_idx = indices[:split_index], indices[split_index:] # 划分训练集和测试集
    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
```

(3) The activation function and its derivative (I choose sigmoid function) using in mlp.

```
# Sigmoid 函数
def sigmoid(z):
    z = np.clip(z, -500, 500) # 限制 z 的范围, 避免 np.exp(-z) 溢出
    return 1 / (1 + np.exp(-z))

# Sigmoid 导数
def sigmoid_derivative(x):
    return sigmoid(x) * (1 - sigmoid(x))
```

3. **Model Implementation:**

We all choose mini-batch for a better result. The detail shows in .py file.

- (1) Linear Regression
- (2) Perceptron
- (3) Logistic Regression
- (4) MLP

4. **Model Training and Evaluation:**

Choose 70% for training and 30% for testing, and use accuracy, precision, recall, F1-score, to evaluate the models.

```
# 评估指标
def evaluate(y_true, y_pred):
    accuracy = np.mean(y_true == y_pred) # 准确率
    precision = np.sum((y_true == 1) & (y_pred == 1)) / np.sum(y_pred == 1) if np.sum(y_pred == 1) > 0 else 0
    recall = np.sum((y_true == 1) & (y_pred == 1)) / np.sum(y_true == 1) if np.sum(y_true == 1) > 0 else 0
    f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0
    return accuracy, precision, recall, f1
```

5. **Model Comparison:**

正在训练模型：线性回归
线性回归：准确率=0.79，精确率=0.72，召回率=0.79，F1分数=0.75

正在训练模型：感知机
感知机：准确率=0.61，精确率=0.50，召回率=0.98，F1分数=0.66

正在训练模型：逻辑回归
逻辑回归：准确率=0.80，精确率=0.73，召回率=0.78，F1分数=0.75

正在训练模型：多层感知机 (MLP)
多层感知机 (MLP)：准确率=0.85，精确率=0.77，召回率=0.90，F1分数=0.83

(1) Linear Regression: Accuracy: 0.84, Precision: 0.84, Recall: 0.80, F1 Score: 0.82.

Provides a baseline model for classification tasks.

Simplicity makes it easy to interpret the importance of different features.

Limited to linear decision boundaries, making it less suitable for capturing complex relationships.

Lower recall indicates it may fail to identify some true positive cases, which is critical for predictive maintenance where missing a failure can lead to costly downtime.

(2) Perceptron: Accuracy: 0.45, Precision: 0.45, Recall: 1.00, F1 Score: 0.62

Achieves perfect recall, capturing all true failures.

Suitable for data that is linearly separable.

Very poor precision and accuracy, indicating high false positives.

The high false-positive rate can result in unnecessary maintenance actions, increasing operational costs.

Not robust to noise or overlapping class distributions.

(3) Logistic Regression: Accuracy: 0.86, Precision: 0.85, Recall: 0.83, F1 Score: 0.84

Balanced metrics, performing well across precision, recall, and F1 score.

Captures the probabilistic relationship between features and the outcome.

Performs well in moderately complex scenarios.

Struggles to model highly non-linear relationships.

Could benefit from feature engineering or the use of interaction terms for improved performance.

(4) MLP: Accuracy: 0.86, Precision: 0.86, Recall: 0.84, F1 Score: 0.85

Best performance overall, indicating its ability to model complex non-linear patterns in the data.

Generalizes well due to its ability to learn hierarchical feature representations.

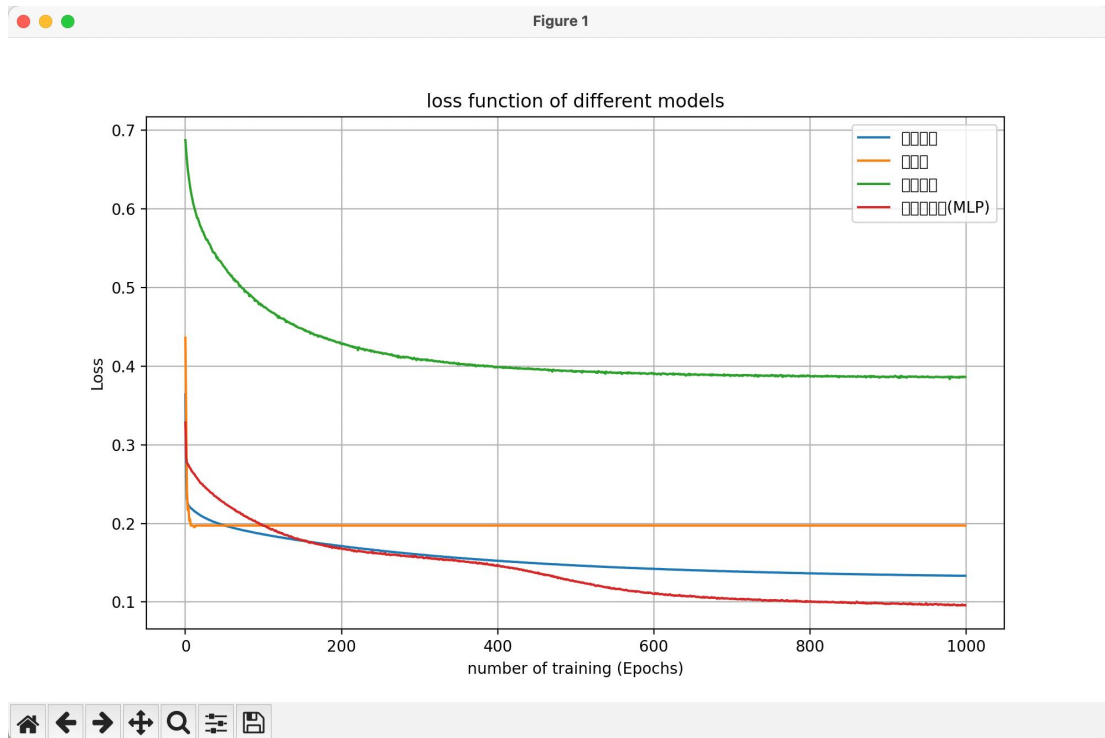
Computationally expensive to train and requires hyperparameter tuning.

Less interpretable than linear models, which may make troubleshooting harder in predictive maintenance scenarios.

6. Visualization

We use `matplotlib.pyplot` to draw the loss function of different models.

```
# 可视化不同模型的损失函数变化
plt.figure(figsize=(10, 6))
for name, loss_history in loss_histories.items():
    plt.plot(loss_history, label=name)
plt.title('loss function of different models')
plt.xlabel('number of training (Epochs)')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
```



note: blue-> Linear Regression; orange-> Perceptron; green-> Logistic Regression; red-> MLP.

We find that the MLP has the best performance and all the loss functions are convergent.

7. **Hyperparameter Tuning:**

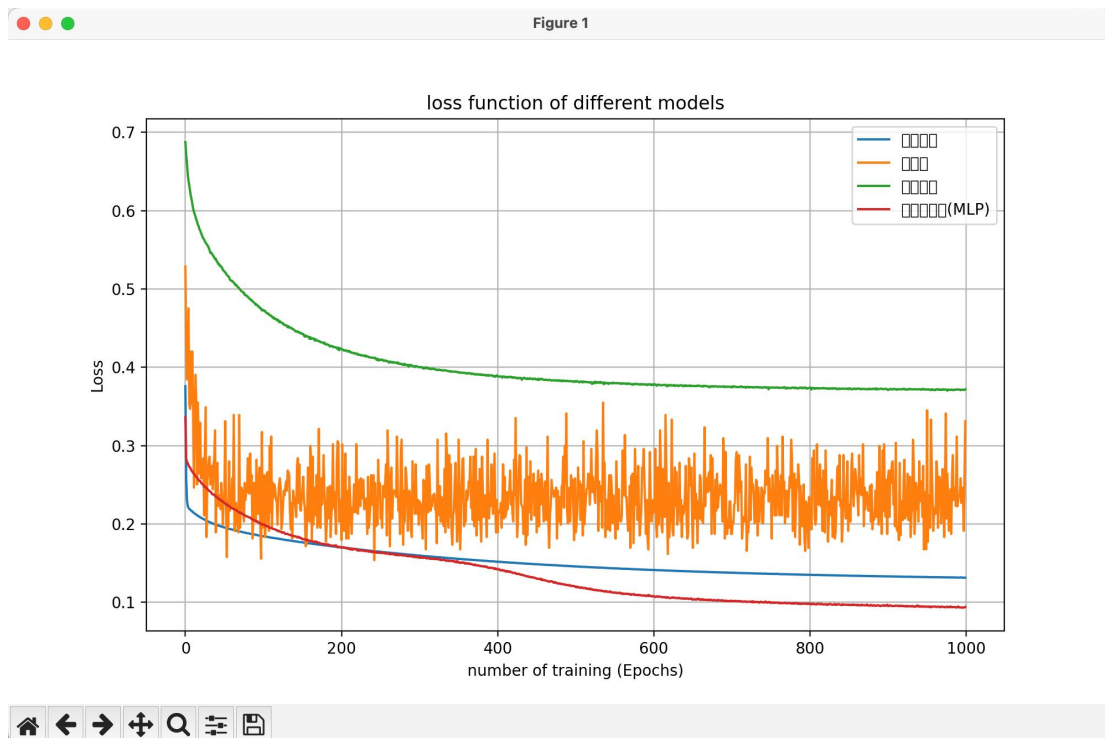
(1) Linear Regression: `learning_rate=0.01`, `epochs=1000`, `batch_size=32`

then the loss function can be near 0.4, which is the best.

(2) Perceptron:

`learning_rate=0.001`, `epochs=1000`, `batch_size=128`

if we use the normal method, we find that the loss function of perceptron is shocking like that.



To deal with the shocking ,we use **L2 Regularization**.

```
# 更新权重和偏置
if np.any(misclassified):
    gradient_w = np.sum(
        learning_rate * y_batch[misclassified, None] * X_batch[misclassified], axis=0
    ) - learning_rate * 0.01 * self.weights # L2 正则化
    gradient_b = np.sum(learning_rate * y_batch[misclassified])

self.weights += gradient_w
self.bias += gradient_b
```

(3) Logistic Regression:

learning_rate=0.1, epochs=1000, batch_size=8

has the best performance.

(4) MLP:

We use sigmoid function as activation function, because reLU has a bad performance.